Project 1 SQL

CSC5041 2026
The deadline for project 1 is: November 15th 23:59:59

1. Aims

This project aims to give you practice in:

- Reading and understanding a moderately large relational schema (MyDATASET).
- Implementing SQL queries and views to satisfy requests for information.

The goal is to build some useful data-access operations on the MyDATASET database. The data may contain some data inconsistencies; however, they will not affect your answers to the project.

2. How to do this project

- Read this specification carefully and completely.
- Familiarise yourself with the database schema (description, SQL schema, summary).
- Make a private directory for this project, and put a copy of the proj1.sql template there.
- You must use the create statements in proj1.sql when defining your solutions.
- Look at the expected outputs in the expected_qX tables loaded as part of the check.sql file.
- Solve each of the problems below, and put your completed solutions into proj1.sql.
- Check that your solution is correct by using the check_qX() functions.
- Test that your proj1.sql file will load without error into a database containing just the original MyDATASET data.
- Double-check that your proj1.sql file loads in a single pass into a database containing just the original MyDATASET data.
- Submit the project via Moodle.
- PL/pgSQL functions are NOT allowed in this project; every solution must be a plain SQL view.
- For each question, you must produce the result within 120 seconds.

3. Introduction

All universities require a significant information infrastructure in order to manage their affairs. DataSet University (DSU) handles its administrative information using a version of the PeopleSoft product called Campus Solutions, accessed via the MyDATASET portal and maintained by the NSS unit. The database behind the system is hosted on a large Oracle server and is referred to throughout as the NSS database. NSS has several deficiencies: it has no representation of degree-program structures, no waiting lists for course or class enrolment, and only poor integration with the DSU Online Handbook. To address these gaps, the MyDATASET data model was devised on top of the core NSS data (students, staff, courses, classes, etc.). The MyDATASET database encompasses the functionality of NSS, the DSU Online Handbook, and the CATS (room allocation) database. The MyDATASET data model, schema and database are described in the description document that accompanies this project.

4. Setting Up

To install the MyDATASET database on your server (Windows/macOS/Linux, using the import method practised in the labs):

```
$ createdb proj1
$ psql proj1 -f MyDATASET.dump
```

If PL/pgSQL is already installed in template1, you will see one message as the database loads: "ERROR: language plpgsql already exists". You can ignore that message, but all other ERROR messages must be investigated. After loading, run the check code and then your solution:

```
$ psql proj1 -f check.sql
$ psql proj1 -f proj1.sql
```

5. Important Advice Before You Start

- The marks reflect the relative difficulty/length of each question.
- Make sure that your queries work on any instance of the MyDATASET schema; do not customise them to work on just this database.
- Do not assume that any query will return a single result; even if phrased as "most" or "biggest", there may be two or more equally big instances.
- When queries ask for names, use the People.name field; it is there precisely to produce displayable names.
- When queries ask for a student/staff ID, use the People.unswid field; People.id is an internal numeric key.
- Unless stated otherwise, the order of tuples in the result does not matter; check.sql sorts your results automatically for comparison.
- The precise formatting of fields matters, e.g. converting a number to a string with to_char may no longer match a numeric field containing the same value.
- You may define as many additional views as you need, provided that the definitions in proj1.sql are preserved.
- Hardcoding is strictly forbidden.

6. Tasks

To facilitate semi-automatic marking, pack all your SQL solutions into a view as defined in each problem (see the proj1.sql template).

Question 1 (4 marks)

Define an SQL view Q1(subject_code):
Give the codes of PG-career subjects that are offered directly by the School of Computer Science and Engineering and whose uoc is greater than 6. PG refers to the Postgraduate career (Subjects.career). Use the longname column of the organisational unit to match the school name.

- subject_code should be taken from the Subjects.code field.

Question 2 (4 marks)

Define an SQL view Q2(course_id) that gives the id of the course that holds Studio classes in at least 3 different buildings.

- course_id should be taken from the Courses.id field.

Question 3 (4 marks)

Define an SQL view Q3(unswid, name):
Display the unswid and name of all distinct staff who taught the student Hemma Margareta.

- unswid should be taken from the People.unswid field.
- name should be taken from the People.name field.

Question 4 (5 marks)

Define an SQL view Q4(facility):
Give a complete list of the facilities that do not exist on any of the buildings on the DSU Kensington campus with a grid reference that begins with the letter "C".

- facility should be taken from the Facilities.description field.
- A building grid reference is stored in the Buildings.gridref column.

Question 5 (5 marks)

Define an SQL view Q5(name, ncourses):
Print the name of the person(s) who has been lecturer-in-charge (LIC) for the largest number of courses, and the number of courses for which they have been LIC. In the database the LIC has the role of "Course

Convenor". It is possible to return more than one lecturer, since they may be in charge of the same number of courses.

- name should be taken from the People.name field.
- ncourses should be returned as an integer.

Question 6 (5 marks)

Define an SQL view Q6(school_name, stream_count):
Give the schools that offer more streams than the School of Computer Science and Engineering. Only consider the streams that are directly offered by an organisational unit that is a school.

- school_name should be taken from the OrgUnits.longname field.
- stream_count should be returned as an integer.

Question 7 (5 marks)

Define an SQL view Q7(subject_code, avg_mark):
Give the subject codes (of the related courses) and the average mark of all PG-career courses offered by the School of Computer Science and Engineering in Semester 1 2010.

- subject_code should be taken from the Subjects.code field.
- Assume that if a student does not have a mark, they have a 0 mark instead.
- Only consider the courses having more than 10 students.
- Use AVG() when calculating the average marks, and save the results as numeric(4,2).

Question 8 (6 marks)

Define an SQL view Q8(subject_code, inc_rate):
Give the subjects in 2008, offered by the School of Chemistry or the School of Accounting, where the average mark received by its students increased the most from semester 1 to semester 2. Give the subject code and the increase rate of its average mark.

- subject_code should be taken from the Subjects.code field.
- Only consider non-null marks.
- Increase rate in average mark = (avg_mark of S2 - avg_mark of S1) / avg_mark of S1.
- Round the increase rate to the nearest 0.0001 (e.g. if inc_rate = 0.01, return 0.0100; if 0.01234, return 0.0123; if 0.02345, return 0.0235).

Question 9 (6 marks)

Below are the rules for a student academic standing:

- i. If a student takes more than one course in a semester, their standing is "Probation" if none of them were passed, "Referral" if 50% or fewer of the taken courses were passed, and "Good" otherwise.
- ii. If a student takes only one course in a semester, they receive "Good" if they pass it, and "Referral" if they fail it.

Define an SQL view Q9(unsw_id, name, academic_standing) giving the unsw_id, name and academic standing of all students whose unswid begins with "313" in semester 2012 S1.

- unsw_id should be taken from the People.unswid field.
- name should be taken from the People.name field.
- academic_standing should be one of "Good", "Probation", "Referral".
- Only consider courses in which the student receives a non-null mark; to pass a course a student needs a mark of 50 or more; do not consider students who never received any mark in that semester.

Question 10 (6 marks)

Define an SQL view Q10(subject_code, year, term, fail_rate):
Give the fail rate for the semester with the largest number of enrolled students for each COMP course, only considering courses with more than 150 students enrolled.

- subject_code should be taken from the Subjects.code field.
- year should be taken from the Semesters.year field.

- term should be taken from the Semesters.term field.
- fail rate = number of students with a mark less than 50 / number of students with a mark.
- When counting enrolled students, consider those with a null mark; but when calculating the fail rate, do not consider null-mark records.
- Round fail_rate to the nearest 0.0001.
- A COMP course refers to a course whose subject code starts with "COMP".

7. Submission

- Submit an electronic copy of your answers to the course website in Moodle.
- The file name must be 226010XXX-Proj1.sql, where 226010XXX is your student ID.
- If you submit more than once, the last submission replaces the previous one.

The proj1.sql file should contain answers to all of the exercises and should be completely self-contained, able to load in a single pass so that it can be auto-tested: a fresh MyDATASET database is created, check.sql is loaded, your proj1.sql is loaded, and each checking function is executed.

8. Check your Answers

```
$ dropdb proj1
$ createdb proj1
$ psql proj1 -f MyDATASET.dump
$ psql proj1 -f check.sql
$ psql proj1 -f proj1.sql
$ psql proj1
proj1=# select check_q1();   ... check your solution to question 1
proj1=# select check_q10();  ... check your solution to question 10
proj1=# select check_all();  ... check all your solutions
```

Notes:

- Your proj1.sql must load and run correctly (no syntax errors) and contain all view definitions in the correct order. If we need to manually fix your file to test it, you will be penalised by half of the mark for each problem.
- Write queries that are reasonably efficient: each question must produce its result within 120 seconds.

9. Late Submission Penalty

5% reduction (-2.5 out of 50) for each 24 hours after the deadline date and time. Submissions more than five days late will not be marked.

10. Use of AI tool

You may use AI tools to help you understand a question or to get a hint. Submitting work that was produced wholly by an AI will be treated as academic misconduct.